

From-scratch implementation attempt of PPO for Mujoco

Only based on the literature available at time of publication, without code reviews

Reinforcement Learning Coffee, SARL PLUS University Salzburg
February 13th, 2026

Adrian Scheerer

adrianscheerer@gmail.com

Background

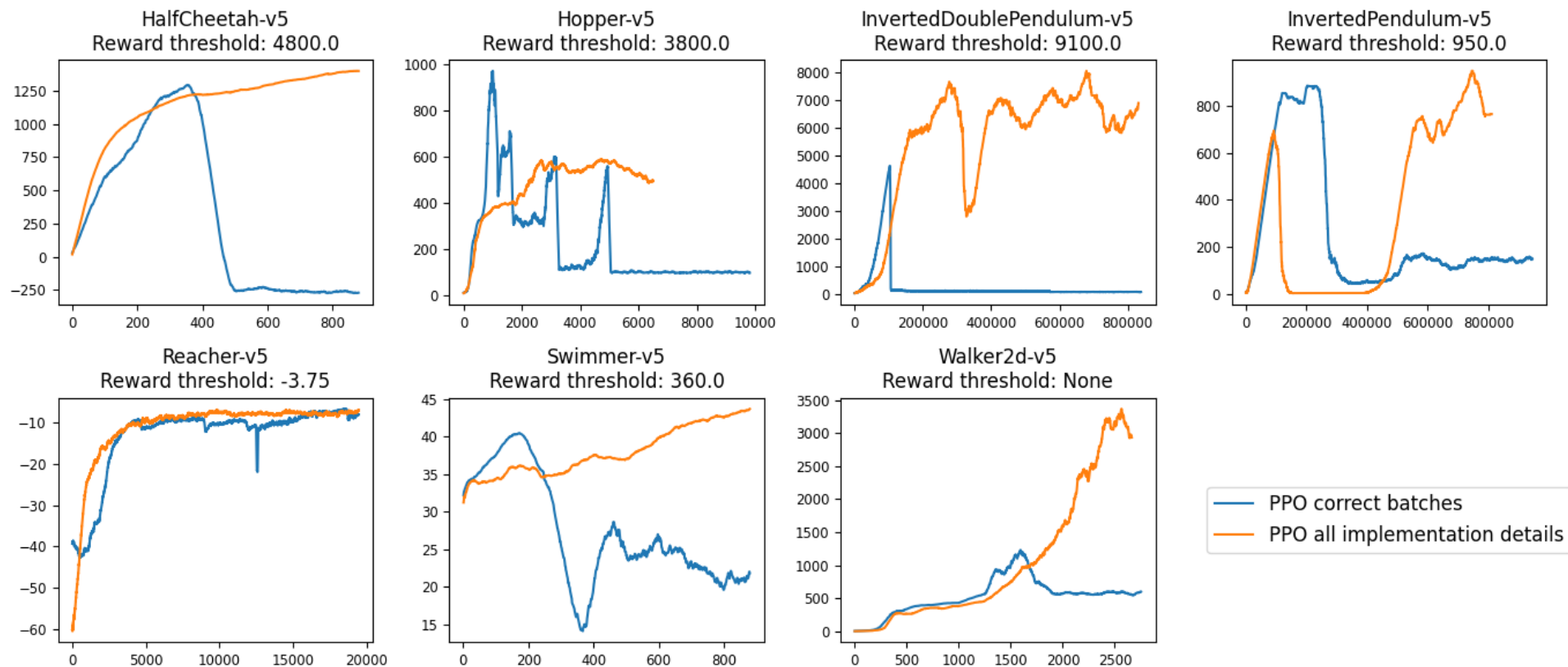
Motivation: A paper on RL for LLM's using only synthetic Python code, mentioning PPO

Can I implement PPO without looking at reference implementations?

Only using resources available at time of publication

1. Went as far as I could based on original literature
2. Compared to "The 37 Implementation Details of Proximal Policy Optimization"
3. Implemented remaining details

Results



"PPO correct batches" is what was possible for me based only on the paper

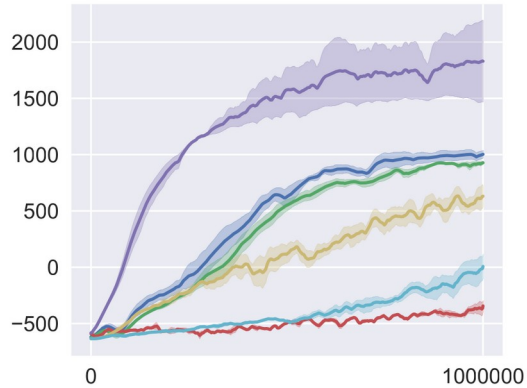
Stochasticity and (non-)repeatability of own results



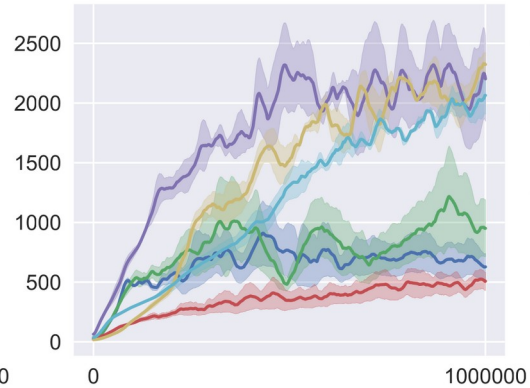
3 runs with different seeds based on own implementation with all details. Not clear what causes collapse for Hopper & Pendulums.

Original results

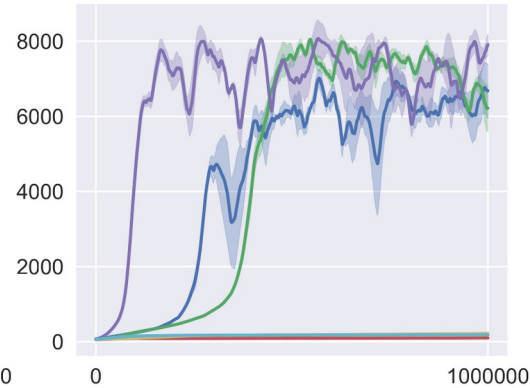
HalfCheetah-v1



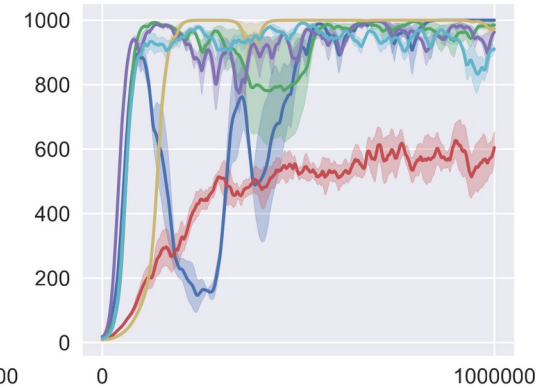
Hopper-v1



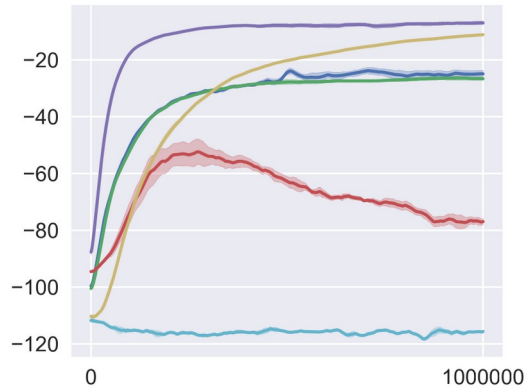
InvertedDoublePendulum-v1



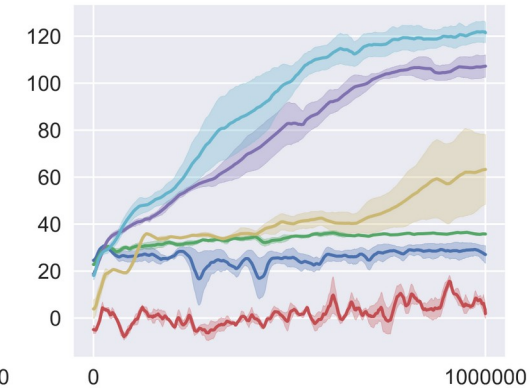
InvertedPendulum-v1



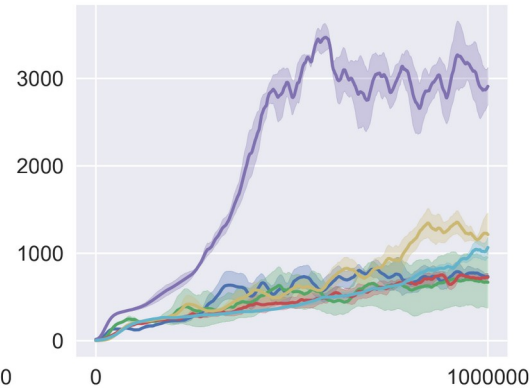
Reacher-v1



Swimmer-v1



Walker2d-v1



- A2C
- A2C + Trust Region
- CEM
- PPO (Clip)
- Vanilla PG, Adaptive
- TRPO

<https://arxiv.org/abs/1707.06347>

Comparison against reference implementation

13 core implementation details	
1. Vectorized architecture	N/A
2. a) Orthogonal Initialization of Weights and Constant Initialization of biases 2. b) Policy output layer weights are initialized with the scale of 0.01	✗
3. The Adam Optimizer's Epsilon Parameter	✗
4. Adam Learning Rate Annealing In MuJoCo, the learning rate linearly decays from $3e-4$ to 0.	✗
5. Generalized Advantage Estimation	✓
6. Mini-batch Updates	✓
7. Normalization of Advantages	✗
8. Clipped surrogate objective	✓
9. Value Function Loss Clipping	✗
10. Overall Loss and Entropy Bonus	N/A
11. Global Gradient Clipping	✗
12. Debug variables	N/A
13. Shared and separate MLP networks for policy and value functions	✓

9 details for continuous action domains	
1. Continuous actions via normal distributions 2. State-independent log standard deviation 3. Independent action components 4. Separate MLP networks for policy and value functions	✓
5. Handling of action clipping to valid range and storage	N/A
6. Normalization of Observation 7. Observation Clipping	✗
8. Reward Scaling 9. Reward Clipping	✗

Review of my implementation “PPO correct batches” against [The 37 Implementation Details of Proximal Policy Optimization](#) which is based on an analysis of the original code in [1] and [2]. Not all details are applicable to Mujoco.

Outcome

Despite not perfectly reproducing the results or solving all environments, I'm reasonably happy with the outcome:

- Great learning outcome anyway - Algorithm trains at all!
- Results at least qualitatively ~roughly in line with reported results, e.g.:
 - Much better than REINFORCE with baseline
 - Get very good rewards for Swimmer-v5 with $\gamma = 0.9999$
- Appreciate nicely structured and fast code
- Have some idea of the hyper-parameters & code structure
- Exercise goes beyond a course on RL and is a good bridge to empirical research

Development best practices

- Write out math by hand: all details, step-by-step (at episode boundaries!), math version, loop/vectorized/batch-vectorized versions. Implement & check each version.
- Single file, few abstractions, yet code separated by purpose (collect data, process it, update)
- Reproducibility of own results? Evolving code may not be reproducible/comparable despite seeds
- Work in teams, spot more errors
- Use LLMs? Off-by-one errors significant. As a learning exercise, did not use any.

Outlook

I have run out of ideas what to debug, except against reference implementations

Algorithmic tweaks can have non-intuitive outcome

Did not attempt implementing PPO modifications for Roboschool & Atari environments without knowing best practices

If you'd like to discuss, happy to talk!